Brighter High Level Design

ARB Signed Off. Minutes pending

Update the warning to a Success Box with the text of "ARB Signed Off" when signed off by ARB, with a link to the minutes for the session it was signed off in.

- Context/Background
- Requirements
  - Non-Requirements/Out of Scope
  - NFR
- Assumptions
- Architecture
  - Why Brighter?
    - 1. Separation of Concerns
    - 2. Encapsulation of Cross-Cutting Concerns
    - 3. Reliability via Outbox and Inbox
    - 4. Durable Execution
    - 5. Transport Flexibility
  - Technologies
  - Architecture Patterns, Principles and Standards
  - Debt Remediation
  - Risks and Issues
  - Design Considerations
- Decisions

Context/Background

*Update the **LPM Reference** in the below Jira Add-in (Set the "Display Options" to "Table" (as this is not the default!). Add "Columns to display" (in this order): Key, Summary, Status, Funded status, EPIC Owner, Architect assigned)*

Key	Summary	Status	Requester	Project Manager	Architect assigned
LPM-934	Contract Pack Vault	CLOSED	Alastair.Waters	Becky Parsons	Dennis Steinmeijer

Synced just now • 1 item

In distributed systems and modern applications, messaging is the backbone of reliable communication between components. Many teams eventually face the same question: should we roll our own lightweight messaging solution, or adopt a library like **Brighter**?

Brighter has been around for over a decade, is open source, and is actively maintained by a group of contributors. While frameworks like **MassTransit** or **MediatR** are well known (with MassTransit recently moving commercial), Brighter offers an OSS-first alternative with a distinct

design philosophy. Understanding the problems Brighter solves, how it solves them, and its trade-offs helps teams decide whether to adopt it—or whether they should even be using a message library at all.

This HLD proposes using Brighter to standardise internal asynchronous communication and durable execution across services, reduce ownership cost versus “rolling our own”, and enable portability between Azure Service Bus (ASB) in cloud and RabbitMQ on-prem.

Key drivers:

- Consistent resilience and reliability (retry, backoff, circuit-breaker, defer, DLQ via broker).
- Transactional messaging (Outbox) and consumer idempotency (Inbox).
- Clear separation of concerns via middleware pipelines (observability, validation).
- Transport abstraction to support ASB ↔ RabbitMQ migration without rewriting handlers (“abstraction, not illusion”).

Scope: internal eventing/command handling between platform and product services; not public event APIs.

Requirements

1. **Transport portability** – Abstract application code from broker specifics; support ASB in cloud and RabbitMQ on-prem with explicit configuration per transport.
2. **Transactional messaging (Outbox)** – Persist outgoing messages in the same transaction as domain state changes.
3. **Consumer idempotency (Inbox)** – De-duplicate message handling to safely support at-least-once delivery and replays.
4. **Durable execution & failure semantics** – Standardise options for retries (exponential + jitter), timeouts, circuit-breakers, and **defer/requeue** for back-pressure; rely on broker DLQ for poison messages.
5. **Dead-letter integration** – DLQ behaviour should align with native broker capabilities. The chosen library should delegate DLQ management entirely to the broker, ensuring that dead-letter routing, retention, and reprocessing follow the broker’s own semantics. The library does not impose an extra abstraction layer or custom logic, avoiding ambiguity and reducing the risk of hidden or hard-to-diagnose failures.
6. **Observability** – Propagate correlation/causation IDs; emit structured logs, traces, metrics (lag, retry count, DLQ rate); provide a replay audit trail.

7. **Ordering & partitioning** – Preserve per-key ordering where required;
8. **Message evolution** – Provide pipelines to allow adding functionality to support major versioning of events and commands; enable validation and safe transforms (compression, encryption, claim-check).
9. **Security** – Support adding encryption at rest/in transit; PII handling via transformers (encrypt/claim-check); secrets via standard platform mechanism.

Non-Requirements/Out of Scope

1. **Long-running workflows** – No general-purpose workflow engine/orchestrator.
2. **Public event API** – No external/public eventing exposure.
3. **Exactly-once delivery** – Not targeted; design assumes at-least-once with Inbox idempotency.

NFR

- **Availability:** Brighter supports high availability through stateless performers that can be scaled horizontally, while actual uptime targets (e.g., ≥99.9%) depend on consumer deployment, broker clustering, and infrastructure SLAs.
- **Latency:** Brighter introduces minimal overhead. Handler latency (P50/P95) is primarily determined by consumer implementation and broker performance; Brighter's single-threaded performers ensure predictable processing.
- **Throughput:** Throughput scales with the number of performers and replicas. Brighter provides back-pressure controls via prefetch and concurrency settings, but final throughput depends on consumer configuration.
- **Durability:** Achieved through the Outbox pattern (transactional message persistence) and acknowledgement-based processing. Brighter, by default, only acknowledges messages if they are successfully processed (this includes being persisted in the outbox) or NAcks them if they are unprocessable and should be sent to the DLQ to ensure they are not lost.
- **Security:** Brighter supports message transformers for encryption, compression, and claim-check, but actual encryption, key management, and PII controls are implemented by consuming teams.
- **Operability:** Built-in middleware hooks for metrics, tracing, retries, and DLQ handling enable observability; consumers must integrate with platform telemetry and runbooks.
- **Scalability:** Horizontal scaling via performers/replicas; consumers define autoscale thresholds and tuning.
- **Compliance:** Brighter supports compliance by design (e.g., secure middleware, idempotency), but alignment to Landmark standards and operational controls remains the

responsibility of the implementing teams.

Assumptions

1. Brighter provides optional sweepers that can periodically delete or archive processed messages from the Outbox and Inbox, helping manage retention and control database growth. However, it does not automatically manage database sizing; consumers must configure and operate these sweepers or implement their own cleanup jobs to ensure tables remain within limits.
2. Third-party dependencies support idempotency or compensations; otherwise, we use Inbox + application-level guards.

Architecture

Initially, the team designated to this project would use Brighter as is and implement and document the following:

- Default configuration (PolicyRegistry, Outbox/Inbox, transports).
- Message envelope conventions and standards.
- DLQ / replay / observability patterns.

These would be used as reference for other teams alongside documented guidance. Later the team can add extension methods with sensible defaults/required minimum settings for the above and distribute it an internal lightweight Nuget package.

Recommendation:

Login to LeanIX

Please make sure you are logged in to your LeanIX Workspace.
Try to reload the page if the issue remains.

Login

Use the Outbox and Inbox patterns by default for services that publish or consume business-relevant messages. This ensures durable, at-least-once delivery with safe replay.

Typical defaults:

- **Producers** of commands/events that reflect business-state changes **SHOULD** use the Outbox.
- **Consumers** who have side effects or need to update persistent state **SHOULD** use Inbox.

Limited exceptions (must be justified):

- **Naturally idempotent processing:** Handlers that are proven idempotent by design (e.g. pure projections that can be rebuilt, or updates that set state to a deterministic function of the message with no external side effects).
- **Best-effort, low-value messages:** Scenarios where occasional loss is acceptable and explicitly agreed (e.g. cache invalidation that can be regenerated).

Components required in Brighter to implement Outbox and Inbox:

- **Outbox (producer side):**
  - An Outbox store (a table in the same database used by the service). Brighter provides the scripts to create the Outbox table.

- An Outbox Dispatcher (typically a background service) that reads unsent messages from the Outbox and publishes them to the broker using Brighter. The dispatcher does the following:
  - Builds message envelops.
  - Runs transformer pipeline (compression, encryption, claim-check).
  - Publishes the message to the broker
  - Handles failures by running Polly policies (retry, timeout, circuit-breaker).
  - Marks successfully sent messages
  - Maintains send order for messages from the Outbox when querying and dispatching in `CreatedAt/Id` order, so that per-key/partition ordering can be preserved by the broker.
- (Optional) Outbox Sweeper or a scheduled cleanup job to enforce retention and control table growth.

- **Inbox (consumer side):**
  - An Inbox store (table or equivalent) that records processed message IDs (and any relevant keys) within a configured deduplication window. Brighter provides the scripts to create the Inbox table.
  - A Performer reads messages from the queue and runs them through the following before applying the handler:
    - Deserialisation.
    - Transformer pipeline (compression, encryption, claim-check).
    - Inbox deduplication checks.
    - Polly policies (retry, timeout, circuit-breaker). This ensures all cross-cutting concerns are enforced consistently.

      This ensures all cross-cutting concerns are enforced consistently.
    - Performers guarantee the following:
      - predictable ordering
      - no threadpool explosion
      - simpler debugging
      - bounded concurrency
  - (Optional) Inbox Sweeper or a scheduled job to remove old entries based on the agreed deduplication window.

Together, these components provide durable, at-least-once delivery with safe replay and idempotent consumption when using Brighter.

Why Brighter?

1. Separation of Concerns

Inline code in controllers or APIs often mixes HTTP concerns (serialisation, status codes) with domain logic (business rules, invariants). Brighter encourages the **Command/Query Separation (CQS) pattern**, separating commands from queries. This aligns well with **hexagonal architecture** (ports and adapters) and simplifies refactoring, testing, and scaling.

2. Encapsulation of Cross-Cutting Concerns

Real-world handlers rarely just contain business logic. They need:

- Resilience (retries, timeouts, circuit breakers)
- Logging and observability
- Validation

Brighter implements these orthogonal concerns as **middleware in pipelines**. This avoids duplicating the same boilerplate across handlers.

3. Reliability via Outbox and Inbox

Messaging systems must cope with crashes, retries, and at-least-once delivery guarantees. Brighter provides:

- **Outbox Pattern**: Ensures messages are persisted in the same transaction as domain changes. Even if the broker is temporarily unavailable, messages won't be lost.
- **Inbox Pattern**: De-duplicates messages on the consumer side, ensuring idempotency in the face of retries or duplicates.

4. Durable Execution

Handlers often need to survive restarts, cope with transient errors, and resume after waiting. Brighter delivers this through a few building blocks that work together:

- **Service Activator & Performers (message pumps):** single-threaded pumps pull → translate → dispatch → *ack only on success*, so a crash before ack leaves the message to be retried.
- **Resilience middleware:** Polly-based **retries with backoff/jitter**, **timeouts**, and **circuit breakers** applied outside your handler code; plus **defer/requeue** to back off without busy looping.
- **Transactional messaging: Outbox** writes the outgoing message in the same transaction as your state change; **Inbox** de-dupes on the consumer so at-least-once delivery won't double-apply side effects.

- **Dead-letter integration:** on terminal failure Brighter **nacks** and lets the broker route to **DLQ** (poison quarantine) instead of maintaining a second error pipeline.
- **Correlation & scheduling:** correlation/causation IDs flow through headers for tracing; optional **scheduled commands** let you pause/wait and resume later.

Together, these give a baseline for **durable, resilient execution**—without pulling in heavyweight workflow engines. When you outgrow this, you can layer lightweight orchestration (e.g., scheduled steps or a small replayer) on top.

5. Transport Flexibility

Many in-house solutions are tightly coupled to a single broker (e.g., RabbitMQ or SQS). Brighter provides an abstraction with pluggable transports (RabbitMQ, Kafka, AWS SQS/SNS, Azure Service Bus, in-memory, etc.), making it easier to switch brokers without rewriting the entire stack.

This is particularly relevant in our context, where services are migrated from Azure Service Bus to RabbitMQ when moved on-premises. Having a library that isolates application code from broker specifics adds significant value. However, this abstraction must be applied with an explicit understanding of the underlying transport. We want **abstraction, not illusion**: Azure Service Bus and RabbitMQ differ in subtle but important ways (delivery semantics, ordering, dead-letter handling, visibility timeouts). If those differences are ignored, subtle bugs and inconsistent behaviour will creep in.

Why Not Role Your Own Implementation?

Rolling your own sounds easy at first, but the complex parts pile up quickly and are easy to get subtly wrong:

- **Transactional messaging (Outbox/Inbox):** ensuring a DB write and a message send are atomic across different systems; de-duping on consume to avoid double side effects.
- **Failure semantics & retries:** distinguishing transient vs terminal faults, adding backoff+jitter, circuit breakers, and avoiding **retry storms** under load.
- **Broker quirks:** handling reconnects, acks/nacks, visibility timeouts, requeue with delay, and offset/sequence management.
- **Message evolution:** schema/versioning, validation, and safe transforms (**compression, encryption, claim-check**) across producers and consumers.

- **Observability:** consistent correlation/causation IDs, structured logs, tracing, metrics for lag, retries, DLQ—plus tooling to investigate and **replay** safely.
- **Operational playbooks:** dead-letter handling, redrive/replay with throttling, and migration between transports without code rewrites.

Missing any of these leads to subtle data loss, duplication, or out-of-order processing—i.e., inconsistent state.

Technologies

Specific technologies not covered by Landmark 3.0 standards or not within the Landmark Technology Radar Mature list of technologies:

- **Brighter (Apache-2)** — mediator + bus; built-in Outbox/Inbox; predictable performers; OSS-first alternative to commercial frameworks.

Architecture Patterns, Principles and Standards

New/clarified patterns needed:

- Standard **Message Formats and Envelopes**. Adopting CloudEvents and AsyncAPI.
- **Replay Runbook**s.
- Outbox.
- Inbox.
- Choreography and Orchestration
- Retry Storms (Thundering Herd)
- Bulkheads

Debt Remediation

Applications in LeanIX track Debt (see: Debt Management on the fact sheets). Describe how this solution impacts Debt Risk, Debt Impact, and Debt Remediation Cost.

Risks and Issues

1. **Transport differences may cause semantic mismatches (ASB vs RabbitMQ).**
   Mitigation: Publish guidance and don't hide away the underlying service bus (“abstractions, not illusions”).
2. **Retry storms under downstream failure.**
   Mitigation: One-layer retries via Polly; exponential + jitter; circuit-breaker.

Design Considerations

1. PolicyRegistry centralisation: standard retry/breaker/timeout keys to avoid nested retries.

Decisions

Keep a record of any decision that were made, when it was made, and who was involved in making it, including any links to Architecture Decision Records.